DATA STRUCTURES " STACKS " C++

Stacks in the Real World

The data structure behind Undo and Redo and inside this app.

1. Why stacks are necessary

A stack is Last-In, First-Out: push adds to the top, pop removes from the top, peek reads the top. That is the whole API and it is exactly what reversing time needs. Undo means: take the most recent change first. No searching, no indexes the top of the stack IS the last thing you did.

2. The two-stack undo/redo design

Keep an UNDO stack and a REDO stack of text snapshots:

Edit: push old text on UNDO, clear REDO.
Undo: push current text on REDO, pop UNDO back into editor.
Redo: push current text on UNDO, pop REDO back into editor.

New typing after an undo wipes REDO the standard behavior of every real editor, and exactly what this app does.

3. How this app uses it

The editor commits a snapshot on every keystroke, so each undo step moves back exactly one character. Toolbar buttons commit instantly. The Stacks tab shows both stacks live with the TOP cell highlighted, plus depth badges on the Undo/Redo buttons. History is capped at 100 snapshots. **Keyboard: Ctrl+Z undo, Ctrl+Y or Ctrl+Shift+Z redo.**

All operations are O(1): push, pop and peek never scan the history.

DATA STRUCTURES " STACKS " C++

Six places C++ programs use stacks

1. Function calls the call stack stores return addresses and locals.
   Deep recursion overflows it: stack overflow.
2. Undo/redo two std::stack<std::string> snapshots, as in stackeditor.cpp.
3. Expression evaluation infix to postfix, bracket matching, calculators.
4. Backtracking maze solving, DFS, and parsers push choices, pop to retry.
5. Browser / editor history back and forward are two stacks.
6. Syntax checking compilers match every opening brace with a stack.

Try it in C++ (stackeditor.cpp ships with this project)

g++ -std=c++17 -O2 -o stackeditor stackeditor.cpp && ./stackeditor

The demo types three phrases, undoes twice, redoes once, then types again to show redo being cleared the same rule the web app enforces.

Cost of every stack operation: O(1) time, O(n) memory for n snapshots.

Takeaway: when the last thing in must be the first thing out
function returns, undos, backtracking, bracket matching reach for a stack.